

Kubernetes Fundamentals for AI Workloads

Session 1 · Week 3 · AI Systems Engineering

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

Recap: Where Week 2 Left Off

One Machine

We containerised GPU workloads with Docker: images, layer caching, CUDA base images, reproducible builds.

One GPU, Many Jobs

Session 4 covered sharing a single GPU: `CUDA_VISIBLE_DEVICE` S, time-slicing, MPS, and MIG partitions.

The Ceiling

Every technique assumed one host. Real training and serving needs many hosts, many GPUs, many teams.

The question this week answers

- Week 2 scheduled work on a single Docker host — you were the scheduler
- Week 3 hands that job to Kubernetes: many nodes, many GPUs, many tenants

The Problem: One Node Isn't Enough

Capacity

A single training job needs 8, 64, or 512 GPUs across many physical hosts — not just one box.

Sharing

Multiple teams compete for the same GPU pool. Someone has to enforce fairness and prevent starvation.

Resilience

Hosts fail, jobs crash, nodes get drained for maintenance. Work must be rescheduled automatically.

If you did all of this by hand...

```
# manual, brittle, does not scale
ssh gpu-node-07 'docker run --gpus=4 ...'
ssh gpu-node-12 'docker run --gpus=4 ...'
# node-07 just died — now what?
# who is tracking which team used which GPU?
```

This is exactly the gap Kubernetes closes: a control system that places, watches, and reschedules work across a fleet.

What Is Container Orchestration?

A definition, in plain language

Orchestration is software that decides where your containers run, keeps them running, and reacts automatically when reality changes — a node fails, load spikes, a job finishes.

You declare

"I want 3 replicas of this container, each with 1 GPU."

It watches

Continuously compares desired state to actual cluster state.

It reconciles

Starts, stops, or moves containers until reality matches your declaration.

What Is Kubernetes?

Kubernetes ("K8s" — 8 letters between K and s) is an open-source system for automating deployment, scaling, and management of containerized applications.

Origin

Google's internal Borg system, open-sourced in 2014.

Governance

Donated to the CNCF in 2015 — vendor-neutral, community-run.

2026 baseline

Current stable line is v1.36; this course targets that release.

Why it won: a large ecosystem (Helm, Argo, Kubeflow, Ray), broad cloud support, and an API extensible enough to model almost any workload — including GPUs.

Kubernetes vs. Alternatives

	Docker Compose	Docker Swarm	Nomad	Kubernetes
Scope	Single host	Small clusters	Multi-workload	Large clusters
GPU scheduling	Manual	Basic	Plugin-based	Native + DRA
Ecosystem	Minimal	Small	Moderate	Largest (CNCF)
Multi-tenancy	None	Limited	Namespaces	RBAC + quotas

Not a knock on the alternatives — Compose and Nomad are great at smaller scope. K8s wins once you need multi-team, multi-GPU, self-healing infrastructure.

Why Kubernetes for AI Infrastructure

Scale

Orchestrate hundreds of GPU nodes.
Autoscale based on queue depth.

Isolation

Namespace + RBAC separation. Each team gets fair-share GPU quota.

Reproducibility

Containerised workloads. Same image from laptop to multi-node cluster.

The industry standard for ML platform teams: MLflow, Kubeflow, Ray, and Argo all run on top of K8s.

K8s in 2026

- K8s 1.33 ships with Dynamic Resource Allocation (DRA) v1beta2 — richer GPU/NIC composability
- Karpenter v1.3 supports Blackwell B200 GPU shape provisioning on major cloud providers

Kubernetes Architecture Refresher

CONTROL PLANE

API Server

Gateway for all control-plane ops. Every kubectl, webhook, and controller goes through it.

etcd

Distributed K/V store. Source of truth for all cluster state. Needs HA in production.

Scheduler

Matches pods to nodes based on resource requests, affinity, and taint/toleration rules.

DATA PLANE (Worker Nodes)

kubelet

Node agent: starts containers, reports node status.

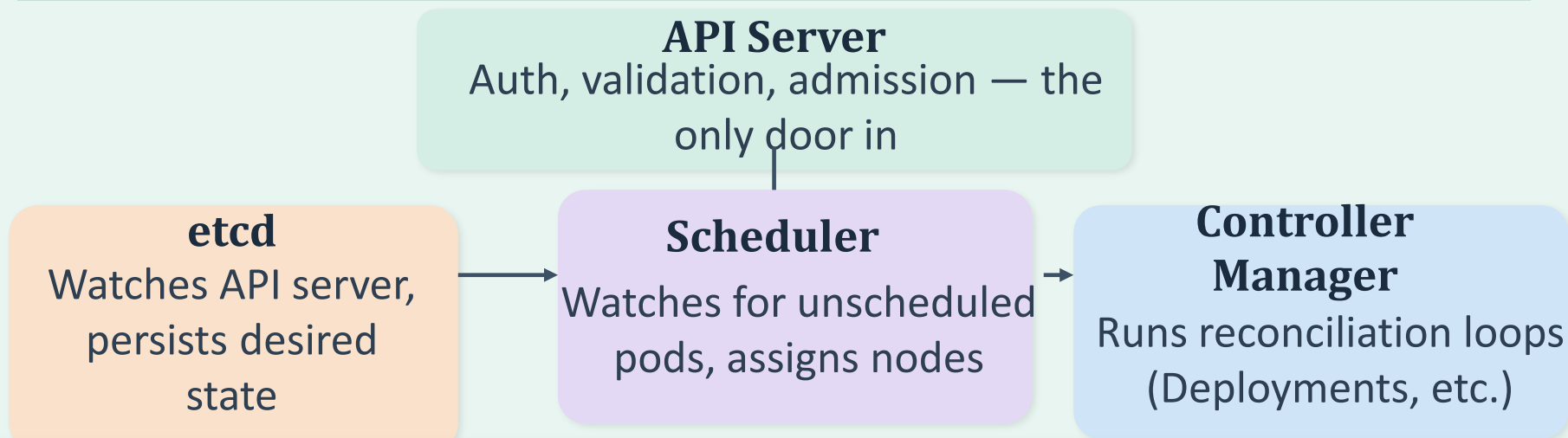
kube-proxy

Service networking: iptables/eBPF rules for ClusterIP.

Container Runtime

containerd 2.0 (CRI). Pulls images, runs pods.

Control Plane Deep Dive



The control loop, in one sentence

- Every controller repeats: observe current state → compare to desired state → take action to close the gap
- The Scheduler is just one controller: it watches for pods with no node assigned and picks one
- The Controller Manager runs many such loops in parallel — Deployment, ReplicaSet, Node, and more

Data Plane Deep Dive

Worker Node

kubelet

Polls API server,
starts/stops
containers via CRI,
reports health



kube-proxy

Programs
iptables/eBPF so
Services route to live
pod IPs



containerd

CRI-compliant
runtime: pulls
images, runs the
actual containers

Pods run here as one or more containers, sharing a network namespace and optional volumes.

GPU-specific job

- kubelet also loads the NVIDIA device plugin, which advertises `nvidia.com/gpu` as a schedulable resource
- containerd hands off to the NVIDIA container runtime to expose the physical device inside the container

Anatomy of a kubectl apply Request



- kubectl sends the pod manifest as an HTTPS request to the API Server
- API Server authenticates you, validates the YAML, runs admission webhooks
- The object is persisted to etcd as the new desired state
- Scheduler notices an unscheduled pod, picks the best-fit node, writes that back
- kubelet on that node sees the assignment, pulls the image via containerd
- kubelet reports Running status back through the API Server; you see it in `kubectl get pods`

Core Kubernetes Objects

Pod	<i>runs your container</i>	Smallest deployable unit. One or more containers sharing network & storage.
Deployment	<i>keeps N pods running</i>	Manages a ReplicaSet. Declarative rollouts and rollbacks.
Service	<i>networking abstraction</i>	Stable ClusterIP / NodePort / LoadBalancer for a pod set.
ConfigMap / Secret	<i>config & credentials</i>	Decouple configuration from image. Secrets base64-encoded (use external-secrets for prod).
PersistentVolumeClaim	<i>storage request</i>	Request durable storage. Bound to a StorageClass (NFS, Ceph, CSI driver).

Pod Anatomy

Pod (shared network namespace + volumes)

Container: pytorch

Main process. Requests
nvidia.com/gpu: 1

shared
localhost
↔

Container: log-shipper

Optional sidecar. Shares
localhost + volume with main
container

IP address

One pod IP shared
by every container
inside it.

Storage

Volumes mount into
any container in the
pod.

Lifecycle

Containers in a pod
start/stop together.

Deployments, ReplicaSets & Rollouts

Deployment

You edit this



ReplicaSet

Auto-generated per
version



Pods (x N)

Actual running replicas

Rolling update, step by step

- You change the container image tag on the Deployment
- A new ReplicaSet is created; old one scales down as new one scales up, gradually

```
kubectl create deployment inference \
  --image=nvcr.io/nvidia/pytorch:25.04-py3 --replicas=3
kubectl set image deployment/inference \
  pytorch=nvcr.io/nvidia/pytorch:25.05-py3# triggers rollout
kubectl rollout status deployment/inference
kubectl rollout undo deployment/inference# instant rollback
```

Services & Networking Types

ClusterIP

Internal-only virtual IP. Default type — pod-to-pod traffic within the cluster.

NodePort

Opens a static port on every node. Simple, but rarely used in production directly.

LoadBalancer

Provisions a cloud load balancer that routes external traffic in.

Client



Service
stable virtual IP



Pod (any replica)
IP changes on restart

A Service is the stable front door in front of pods whose IPs are constantly changing as they're rescheduled.

GPU Resource Model in Kubernetes

Device Plugin (legacy)

```
resources:  
  limits:  
    nvidia.com/gpu: 1
```

NVIDIA device plugin advertises GPUs as countable resources. Integer only — no fractional GPU without MIG or time-slicing.

DRA (K8s 1.31+ / 2026)

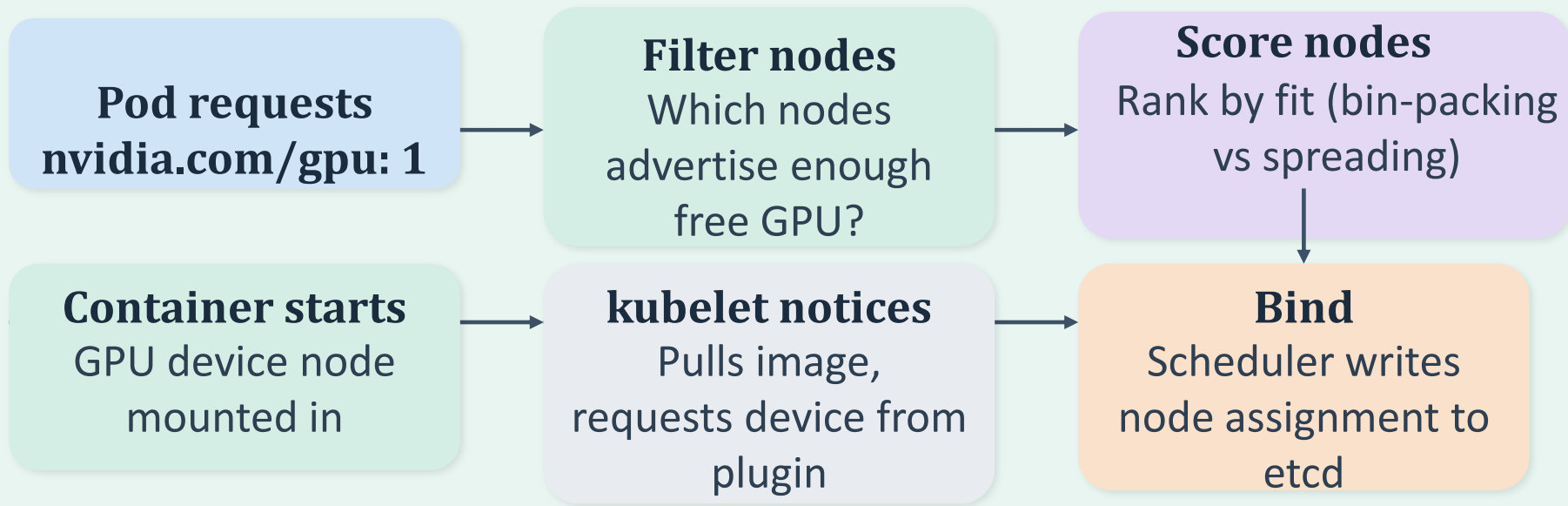
```
resourceClaims:  
- name: gpu-claim  
  resourceClaimTemplateName:  
    my-gpu-template
```

Structured parameters, composite devices (GPU + NIC), and partial-device claims via MIG slices.

Multi-Instance GPU (MIG) — Blackwell B200

- B200 supports up to 7 MIG slices (1g.23gb to 7g.160gb profiles)
- Expose individual slices as separate K8s resources: `nvidia.com/mig-1g.23gb`
- Time-slicing (shared access): `nvidia-device-plugin 0.17+` `time_slicing` config

Scheduler Decision Flow for GPU Pods



If no node has a free GPU...

- Pod stays in Pending — check with `kubectl describe pod` (Events section)
- Cluster autoscaler / Karpenter can provision a new GPU node on demand (v1.3 supports B200 shapes)
- Without autoscaling, the pod waits indefinitely for capacity to free up

Anatomy of a GPU Pod Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: pytorch-gpu-smoke
  labels:
    app: pytorch-test
spec:
  runtimeClassName: nvidia      # use NVIDIA container runtime
  tolerations:
  - key: nvidia.com/gpu
    operator: Exists
    effect: NoSchedule
  containers:
  - name: pytorch
    image: nvcr.io/nvidia/pytorch:25.04-py3  # PyTorch 2.11 / CUDA 13.0
    resources:
      limits:
        nvidia.com/gpu: "1"
      command: ["python", "-c",
        "import torch; print(torch.cuda.get_device_name(0))"]
    restartPolicy: Never
```

YAML Manifest, Field by Field

```
runtimeClassName:  
nvidia  
tolerations:  
[nvidia.com/gpu:  
Exists]  
resources.limits.nvidi  
a.com/gpu: "1"  
image:  
nvcr.io/nvidia/pytorch  
:25.04-py3  
restartPolicy: Never
```

Tells kubelet to hand off to the NVIDIA container runtime instead of the plain OCI runtime.

Required — this taint sits on every GPU node (see slide 20). Without it, the pod can't land there.

The actual GPU request. This is what the scheduler filters and scores nodes on.

Pinned to PyTorch 2.11 / CUDA 13.0 — matches this course's mid-2026 baseline exactly.

Smoke tests shouldn't restart forever on failure — Never surfaces the error immediately.

Node Taints, Tolerations & Affinity

Taints repel pods. Tolerations allow pods to land on tainted nodes. Affinity attracts pods to specific nodes.

Taint GPU Nodes

```
# Label & taint a GPU node
kubectl label node gpu-node-01 \
    accelerator=nvidia-rtx3050
kubectl taint node gpu-node-01 \
    nvidia.com/gpu=present:NoSchedule
```

Toleration + Affinity

```
tolerations:
- key: nvidia.com/gpu
  operator: Exists
  effect: NoSchedule
affinity:
  nodeAffinity: {required:
    {...}}
```

Best practice: taint ALL GPU nodes. Every GPU workload must tolerate the taint explicitly — prevents CPU workloads from accidentally consuming GPU quota.

Scheduling Decision Diagram

Pod submitted

**Any pod may
schedule here**

**Does node have a
matching taint?**

**Pod must have
matching
toleration**

**Then affinity
rules apply**

Result: only GPU-requesting pods with the right toleration land on GPU nodes; everything else is filtered out first.

Optional:
prefer/require
specific node labels

Namespaces, RBAC & Resource Quotas

Namespace

Logical partition of cluster resources. Convention: one namespace per team or project.

ResourceQuota

Limits total CPU/memory/GPU across all pods in a namespace. Enforces fair-share.

LimitRange

Sets default and max resource requests per pod. Prevents runaway single pods.

RBAC Role/ClusterRole

Controls who can create/delete pods, secrets, PVCs within a namespace.

Multi-Tenant Cluster, End to End

One Kubernetes Cluster

ResourceQuota: 2 GPUs
RoleBinding: alpha-devs
namespace: team-alpha

Pod
1 GPU

Pod
1 GPU

ResourceQuota: 2 GPUs
RoleBinding: beta-devs
namespace: team-beta

Pod
1 GPU

quota
headroom
1 GPU free

*team-beta cannot exceed its 2-GPU quota even if team-alpha's nodes are idle
— quotas are per-namespace, not cluster-wide pooled by default.*

1. As in the Vs Code file

Key Terms Glossary

Node

A physical or virtual machine running kubelet.

Control loop

Observe → compare → reconcile, repeated forever.

DRA

Dynamic Resource Allocation — structured, composable device claims.

Pod

Smallest deployable unit; one or more co-located containers.

Taint / Toleration

Repel-by-default / opt-in-to-land-here pairing.

Namespace

Logical partition of one cluster's resources.

Quick reference — worth pausing on camera so attendees can screenshot before the labs.

Session 1 Summary

- Kubernetes orchestrates AI workloads at scale — isolation, reproducibility, autoscaling
- Control plane (API Server, etcd, Scheduler) + data plane (kubelet, kube-proxy, containerd)
- GPU resources exposed via Device Plugin (nvidia.com/gpu) or DRA v1beta2 (K8s 1.31+)
- B200 MIG slices: up to 7 independent GPU instances from a single physical card
- Taints + tolerations prevent CPU workloads from consuming GPU quota
- Namespaces + ResourceQuota + RBAC = foundation of multi-tenant AI cluster governance